Exercise 4 – Ultrasound fields

In this exercise we will work with an advanced ultrasound simulation tool to investigate interference patterns and the unfocused ultrasound beam profile. We will also compare are results to analytical solutions.

Main learning objectives:

- To learn about ultrasound fields and interference patterns
- To learn about the unfocused beam profile from a ultrasound transducer

Practical skill development:

- To get started with ultrasound field simulations using k-wave
- To make code for analysis beam profiles (depth profile and lateral beam profile)

We will be using a simulation tool called *k-wave*, which is available for both Matlab and Python.

Simulations

K-wave is an advanced simulation program for general acoustic and elastic wave propagation simulations. It was originally developed in Matlab and will probably be most robust in this environment, but I have been using it in Python lately and it should work well for our purpose. I have made the boilerplate code to run a simple simulation as a starting point, and you will mainly modify the simulation setup to reach the learning objectives.

- You can find the Matlab toolbox here: K-wave toolbox link
    - To install unzip somewhere and add the main directory to your Matlab path
- You can find the Python toolbox here: https://github.com/waltsims/k-wave-python
    - To install, make a blank environment and run *pip install k-wave-python*.
    - You may also need to install other basic scientific packages

Quick guide to k-wave:

- In k-wave you setup a simulation *grid*, *source*, *sensor*, and *medium*. See figure 1.
- The simulation is mesh/grid-based, meaning that the spatial and temporal resolution is defined explicitly. The simulation time is very much tied to the grid resolution, and so is the accuracy of the results. The required grid resolution to get sufficient accuracy is already defined in the basic setup.
    - The grid dimensions in k-wave are different from our typical ultrasound setup, the x-axis is our depth (z) axis, and y / z are our lateral dimensions x / y.
    - The grid in k-wave is symmetric in space from -X to +X etx. For a
- Depending on the setup and laptop you have, the simulations can take some time but should not extend beyond some minutes.
- You can reduce the accuracy to more quickly generate results initially and make a more detailed / accurate simulation in the end for the final result.
- The source and sensor definition are tied to the grid resolution. For instance, if the grid resolution is 1/5 of a wavelength in x, y, z directions, then we can define a source or sensor by building up blocks of 1/5 wavelength in size. Usually, we define an empty mask (matrix of zeros) of equal size to the simulation grid and fill in ones where we would like to have a source. We can add different temporal signal sources any source point.
- The sensor is defined in a similar way. To record the signal at all field grid points, for instance to make beam profile plots, simply make a sensor mask matrix which has only ones. If you want to make a transducer, only add sensor points in a given region to mimic this situation.
- We can define what we want to record for the sensors, which can be the pressure field over time, or statistics of the pressure field, such as the maximum or root-mean-square value over time at a given point.
- The manual of k-wave can be found here:
    - Matlab: http://www.k-wave.org/manual/k-wave_user_manual_1.1.pdf
    - Python: https://k-wave-python.readthedocs.io/en/latest/

For the simulation exercises you can use the template code which setups up the main simulation parameters and some boilerplate code. If you want to make movies of the simulations from Python, I find it is best to use *ffmpeg* (needs to be installed).

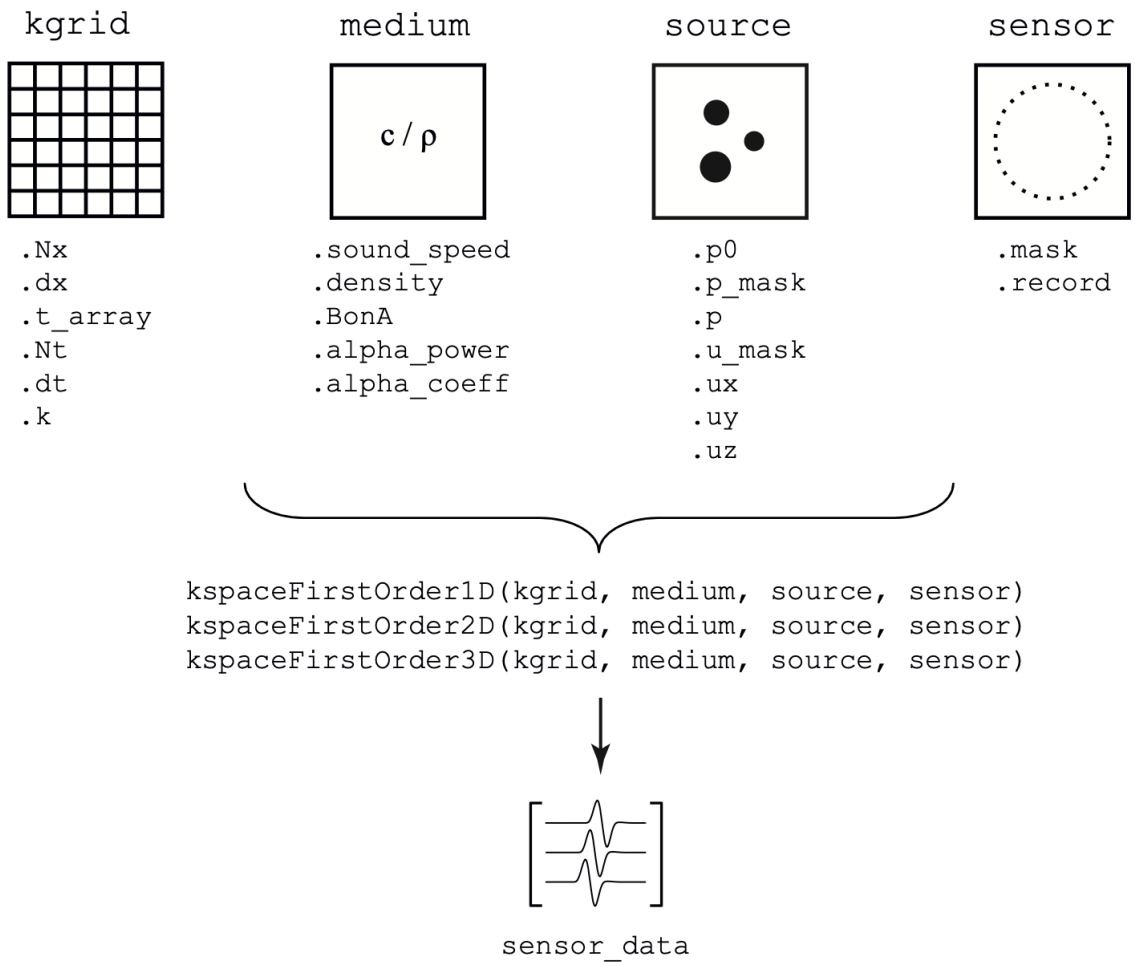


Figure 1: The typical workflow in k-wave, setting up your simulation grid, medium properties, source distribution in space and time, and sensor for registering measurements. Simulations can be run in 1D, 2D or 3D as needed.

1. Point source simulation

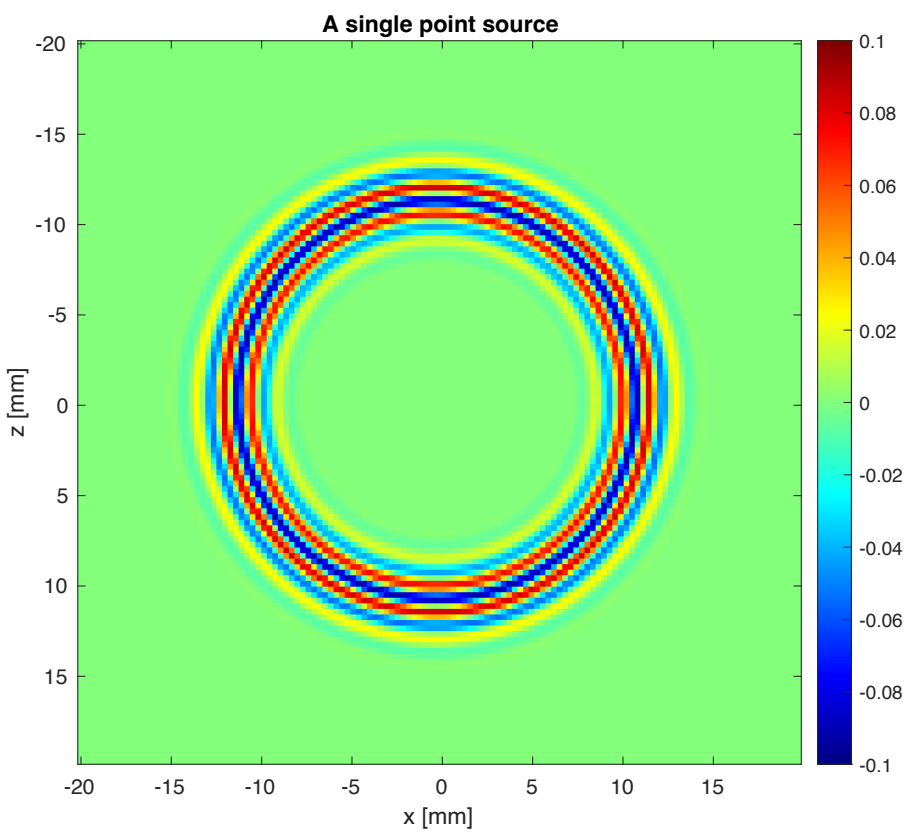
Let's get to know k-wave and ultrasound simulations by first making a simple point pressure source simulation and visualize the result.

Open Exercise_4_fields (.m or .py) to look at the code.

In this example we initially:

- Setup a Gaussian ultrasound pulse with f0 = 1MHz with 10 cycles
- Setup a simulation grid size and spacing corresponding to 5 grid points per wavelength in both x and z
- Setup medium with a homogeneous speed of sound
- Setup our source to be a single point source in the center of the grid
- Setup our sensor to record both the pressure over time, and maximum pressure over time for each point in the grid
- Setup a 2D simulation to save simulation time

Run the example, the output should look something like this (Matlab):



Let's play a bit with the parameters for this simulation example:

- Change the time point (frame) that you visualize to see the progression from a tiny point source to a propagating wave
- Change the number of cycles in the pulse and observe results. You can simulate a continuous wave by extending the number of periods to be high (e.g. 100 - 500).
- Change the grid resolution through the *ppw* parameter (points per wavelength) to see how lower and higher values influences the results and running time of the simulation. *What do you observe?*
- Create a region with a different speed of sound by modifying the medium. For instance, let half the domain have half the speed of sound. What happens?

Just for fun (optional): You can extend this example by adding more scatterers randomly positioned in the grid to make a raindrop simulation! (The *toneBurst* function can also add a delay to the individual signals if you want to).

2. Interference patterns

Let's see what happens when we increase the size of the source. Use a grid resolution of 1/5 wavelengths (ppw=5) and increase the number of cycles in the pulse to 100 to see (more of) the spatial interference pattern.

a) Position 2 point sources next to each other in the y-direction
b) Position 3 point sources next to each other in the y-direction
c) Position 4 point sources next to each other in the y-direction
d) Position 5 point sources next to each other in the y-direction, this should be one wavelength in width

Visualize the resulting interference patterns from a-d. What do you observe as the source increases relative to the wavelength?

3. An unfocused beam profile simulation

Let's use the Huygens principle to simulate an *unfocused ultrasound beamprofile*, here as a simple line source (line of neighboring point sources). Let's use a cardiac transducer as an example but with a smaller aperture to save simulation time.

Notes:

- Starting setup: source_aperture=1cm, source_f0=1e6, source_cycles = 5, grid depth =10cm, grid width = 4cm
- When placing the line source in the domain you can choose a small index, e.g. 10 in the depth direction so that we have most of the depth to analyze our emitted beam profile while reducing simulation time.
    - Note that if you position the source too close to the edges there will be an error in the results (due to interaction with the so-called PML simulation layer just outside the domain, which absorbs the incoming energy).
- The results from the line source will be mirrored at the depth you choose, but we focus on the field underneath the aperture.
- You only need to simulate once, then change the time for each plot in Matlab or a Jupyter Notebook.
- You can set the ppw=10 to get nicer plots when you are happy with the results. This may increase the simulation time substantially depending on your setup (and computer), but should be OK with the setup proposed.

a) Simulate the time-varying pressure field emerging from this source. Visualize the results at different time steps. What do you observe as the pulse propagates?

b) Plot an image of the beam profile using the p_max output from the simulation. This plol is most often shown in dB scale (i.e. 10*log10(p^2)). It is useful to plot the image normalized at each depth when analyzing the beam profile shape. Here is example of Matlab code to do this (before taking the logarithm):

```
% Normalize per depth
p_max_depth = max(p_max');
p_max = p_max ./ p_max_depth';
```

    a. Does the beam profile resemble the theoretical unfocused beam profile shape? Describe why.

c) Plot the *depth profile*, i.e. the depth values *without normalization* along the center of the beam using the p_max output.
    a. In the book Diagnostic Ultrasound Inside Out, on page 176 we see the field for a line aperture like ours. It is here shown that the distance to the maximum pressure is approximately equal to:

$$z_t \approx L_x^2/(\pi\lambda)$$

    Where Lx is our aperture length.

b. Insert your parameters to calculate the distance and indicate in the plot where this distance is. Does it make sense?

d) Change the number of pulse cycles to 200, to approximate continuous wave, and run the simulation and plots again. Compared to the previous beam profile from a short pulse, what changed?

e) Using the continuous wave simulation as in d), plot the lateral beam profile at a given depth. This plot is most often plotted in dB, like the full beam profile.

f) The lateral beam profile in the far field is equal to the Fourier transform of the aperture function. This aperture function is here a line (1D rectangular), and the analytical Fourier transform is a sinc function:

$$\mathrm{sinc}\left(\frac{Lx}{\lambda z}\right)$$

Calculate and plot the theoretical lateral beam profile, compare to the one in d) for different depths, does it match?